

Suricata Dashboard & Scraping

📅 Date July 29, 2026

Rest of the 14 sources:

```
PS F:\CIL Lab\suricata-ids-lab> cd "F:\CIL Lab\suricata-ids-lab\JSON File for Rules\JSON Making"
PS F:\CIL Lab\suricata-ids-lab\JSON File for Rules\JSON Making> python generate_rules_json.py
[+] Fetching OISF index.yaml...
[+] Debug - sources type: <class 'dict'>
[+] Debug - sample keys: ['et/open', 'et/pro', 'oish/trafficid', 'ptresearch/attackdetection', 'scow/enhanced']
[+] Processing source: oish/trafficid -> https://openinfosecfoundation.org/rules/trafficid/trafficid.rules
[->] Captured 34 sample rules from oish/trafficid
[+] Processing source: abuse.ch/sslbl-blacklist -> https://sslbl.abuse.ch/blacklist/sslblacklist_tls_cert.tar.gz
[->] Captured 10225 sample rules from abuse.ch/sslbl-blacklist
[+] Processing source: abuse.ch/sslbl-ja3 -> https://sslbl.abuse.ch/blacklist/ja3_fingerprints.tar.gz
[->] Captured 97 sample rules from abuse.ch/sslbl-ja3
[+] Processing source: abuse.ch/feodotracker -> https://feodotracker.abuse.ch/downloads/feodotracker.tar.gz
[->] Captured 5 sample rules from abuse.ch/feodotracker
[+] Processing source: abuse.ch/urlhaus -> https://urlhaus.abuse.ch/downloads/urlhaus_suricata.tar.gz
[->] Captured 29726 sample rules from abuse.ch/urlhaus
[+] Processing source: etnetera/aggressive -> https://security.etnetera.cz/feeds/etn_aggressive.rules
[->] Captured 24 sample rules from etnetera/aggressive
[+] Processing source: tgreen/hunting -> https://github.com/travisgreen/hunting-rules/raw/master/hunting.rules.tar.gz
[->] Captured 254 sample rules from tgreen/hunting
[+] Processing source: stamus/lateral -> https://ti.stamus-networks.io/open/stamus-lateral-rules.tar.gz
[->] Captured 546 sample rules from stamus/lateral
[+] Processing source: pawpatrules -> https://rules.pawpatrules.fr/suricata/paw-patrules.tar.gz
[->] Captured 21434 sample rules from pawpatrules
[+] Processing source: ptrules/open -> https://rules.ptsecurity.com/files/ptopen.rules.tar.gz
[->] Captured 422 sample rules from ptrules/open
[+] Processing source: aleksibovellan/nmap -> https://raw.githubusercontent.com/aleksibovellan/opensense-suricata-nmaps/main/local.rules
[->] Captured 10 sample rules from aleksibovellan/nmap
[+] Processing source: ipfire/dbl -> https://dbl.ipfire.org/lists/suricata.tar.gz
[->] Captured 56 sample rules from ipfire/dbl
[+] Processing source: julioliraup/antiphishing -> https://raw.githubusercontent.com/julioliraup/Antiphishing/refs/heads/main/antiphishing.tar.gz
[->] Captured 25466 sample rules from julioliraup/antiphishing
[+] Processing source: the-hunters-ledger/open -> https://the-hunters-ledger.com/feeds/suricata/hunters-ledger.rules
[->] Captured 92 sample rules from the-hunters-ledger/open
[+] All target sources matched successfully against the index keys.
[+] Writing 88391 total sample rules to suricata_community_sample.json...
[+] Complete!
PS F:\CIL Lab\suricata-ids-lab\JSON File for Rules\JSON Making>
```

ET:

Optimized code for faster scraping:

1. Doing several SIDs at the same time instead of one at a time

The original script scraped SIDs one by one — open a page, wait for it to load, click through three tabs, grab the info, close the page, then move to the next SID. Every single step waited for the previous one to finish completely.

The fix: instead of one "worker" doing all 71,000 SIDs sequentially, we split the work across **6 workers running at the same time**, each with its own browser tab. While worker 1 is waiting for SID #500's page to load, workers 2-6 are already loading and scraping their own different SIDs in parallel. Think of it like having 6

people scrape instead of 1 — roughly 6x the throughput for the same amount of waiting.

2. Not waiting for the page to be "fully quiet"

The original code used `wait_until="networkidle"` when loading each page — this tells the browser "don't consider the page loaded until there's been no network activity at all for a while." Many modern websites (this one included) keep making small background requests (analytics, ads, polling) that never fully go quiet, so this setting was forcing the script to wait several extra seconds per page for no real benefit.

The fix: switched to `wait_until="domcontentloaded"` — this just waits for the actual page structure (the HTML) to be ready, which is all we actually need since the script already does its own explicit waits for the specific buttons/text it's looking for afterward. This shaves a few seconds off *every single page load*, which adds up hugely over 71,000 pages.

3. Shorter delay between requests, since concurrency already spreads out the load

The original had `time.sleep(2)` after every single SID — a full 2-second pause. With 6 workers now sharing the load, we lowered this to `0.3` seconds per worker. This still avoids hammering the site too aggressively (each worker still pauses briefly), but since 6 workers are pacing themselves independently, the site still sees a steady trickle of requests rather than one single stream — just a faster one overall.

The combined effect

Rough math: if concurrency alone gives ~6x speedup, and faster page loads shave meaningful time off each of those, a task that would've taken ~39 hours sequentially could realistically finish in somewhere around 6-10 hours instead — not a perfect 6x reduction (since the website's own response time is still a shared bottleneck no matter how many workers you throw at it), but a substantial, real improvement.

